

Chapter 6

Evaluating Model Robustness

PCC190 – Secure Machine Learning
Prof. Rodrigo C. P. Silva – DECOM/UFOP

October 2, 2026

Contents

Why evaluation deserves its own chapter	1
1 Metrics for Adversarial Robustness	1
1.1 Budget-based metrics	2
1.2 Distance-based metrics	2
1.3 Cost metrics	2
2 Benchmarking Tools and Frameworks	2
3 Adaptive Attacks: Evaluating Defenses Properly	3
3.1 A recipe	3
3.2 Sanity checks	3
4 Security Evaluations under Different Threat Models	4
5 Interpreting Robustness Evaluation Results	4
6 Hands-On: Setting Up Robustness Benchmarks	5
6.1 Loading models and running AutoAttack	5
6.2 Robustness curve with per-example worst case	5
6.3 Suggested experiments	5
Exercises	6

Why evaluation deserves its own chapter

The history of adversarial ML is largely a history of defenses that were reported as robust and later broken, often within weeks and often by straightforward modifications of known attacks (Carlini and Wagner, 2017; Athalye et al., 2018; Tramèr et al., 2020). The cause was rarely dishonesty. It was the asymmetry of the problem: a defense claim is a universal statement (*no* attack in the threat model succeeds), while an evaluation is a finite set of attacks. The burden is on the evaluator to make that finite set as strong as possible. This chapter covers what to measure, which tools to use, how to build adaptive attacks, and how to read the numbers. The main reference is Carlini et al. (2019).

1 Metrics for Adversarial Robustness

Throughout, $\{(\mathbf{x}_i, y_i)\}_{i=1}^N$ is a test set, $\mathcal{A}$ an attack returning $\mathbf{x}'_i = \mathcal{A}(\mathbf{x}_i, y_i)$ within the threat set $\mathcal{B}_p(\mathbf{x}_i, \epsilon)$, and f the evaluated model (including any preprocessing or randomness).

1.1 Budget-based metrics

Definition 1.1 (Empirical robust accuracy).

$$\text{RA}_{\mathcal{A}}(\epsilon) = \frac{1}{N} \sum_{i=1}^N \mathbb{I}[f(\mathbf{x}_i) = y_i \wedge f(\mathcal{A}(\mathbf{x}_i, y_i)) = y_i].$$

For a set of attacks $\{\mathcal{A}_1, \dots, \mathcal{A}_J\}$, the *worst-case* robust accuracy counts a point as robust only if *every* attack fails on it:

$$\text{RA}_{\text{worst}}(\epsilon) = \frac{1}{N} \sum_i \mathbb{I}\left[f(\mathbf{x}_i) = y_i \wedge \bigwedge_j f(\mathcal{A}_j(\mathbf{x}_i, y_i)) = y_i\right].$$

Per-example worst case is strictly more informative than reporting the minimum of per-attack accuracies, because different attacks succeed on different points. The true robust accuracy $\text{RA}^*(\epsilon)$ (no adversarial example exists) satisfies

$$\text{CA}(\epsilon) \leq \text{RA}^*(\epsilon) \leq \text{RA}_{\text{worst}}(\epsilon) \leq \text{RA}_{\mathcal{A}_j}(\epsilon), \quad (1)$$

where $\text{CA}(\epsilon)$ is a certified accuracy, when available (Chapter 5).

Attack success rate (ASR). Fraction of initially correct points that become misclassified (or reach the target, for targeted attacks). Always state the denominator: “all points” and “initially correct points” give different numbers.

Robustness curves. Report $\text{RA}(\epsilon)$ over a range of ϵ , not a single value. A curve that does not reach zero for large ϵ signals a flawed evaluation (Section 3). The area under the curve summarizes it in one number.

1.2 Distance-based metrics

Minimum-norm attacks (C&W, DeepFool, FAB, [Croce and Hein, 2020a](#)) return, for each $\mathbf{x}_i$, the smallest perturbation $\hat{\rho}_i$ they found. Since any attack finds a perturbation at least as large as the true minimum ρ_i^* , we have $\hat{\rho}_i \geq \rho_i^*$. Report the median $\hat{\rho}$ and the empirical CDF; the robustness curve at any ϵ is the fraction of points with $\hat{\rho}_i > \epsilon$.

Estimates are not lower bounds

Score-based estimates of the minimal perturbation, such as CLEVER ([Weng et al., 2018](#)), use sampling and extreme value theory to estimate a local Lipschitz constant. They are *estimates* and can overstate robustness, especially under gradient masking. Only verification or certification gives lower bounds on ρ^* .

1.3 Cost metrics

For black-box attacks, success should be reported together with the cost: the median number of queries per successful attack and the success rate as a function of the query budget. For poisoning, report CA and ASR versus poisoning rate (Chapter 3). For privacy, report TPR at low FPR and, for DP models, the empirical ϵ lower bound (Chapter 4).

2 Benchmarking Tools and Frameworks

Reimplementing attacks is a common source of evaluation bugs. Use maintained libraries, and cross-check with at least two.

AutoAttack in practice. AutoAttack combines attacks with different failure modes: APGD with cross-entropy and with a targeted margin loss handles gradient-based search with adaptive step sizes; FAB finds minimal perturbations; Square Attack is gradient-free and therefore insensitive to gradient masking. It is evaluated in worst-case fashion per example. Its *standard* version is deterministic; the *rand* version uses EOT for randomized defenses.

A standard attack is not an adaptive attack

AutoAttack is a strong *minimum* for evaluation, not a sufficient one. [Tramèr et al. \(2020\)](#) broke thirteen defenses, several of which had been evaluated with strong standardized attacks, by tailoring the attack to

Tool	Purpose
AutoAttack (Croce and Hein, 2020b)	Parameter-free ensemble for ℓ_∞ , ℓ_2 , ℓ_1 : APGD-CE, APGD-T (targeted, margin loss), FAB-T, Square. The standard reporting protocol for image classifiers.
RobustBench (Croce et al., 2021)	Leaderboard and model zoo evaluated with AutoAttack on CIFAR-10/100 and ImageNet, plus common corruptions. Useful for baselines and pretrained robust models.
Foolbox (Rauber et al., 2020)	Many gradient and decision-based attacks with a uniform interface (PyTorch, TensorFlow, JAX).
ART (Nicolae et al., 2018)	Broad toolbox: evasion, poisoning, extraction and inference attacks, plus defenses; supports many model types including tree ensembles.
torchattacks (Kim, 2020)	Lightweight PyTorch implementations of common evasion attacks.
TextAttack (Morris et al., 2020)	NLP attacks built from goal, constraints, transformation and search (Chapter 7).

Table 1: Common evaluation tools.

each defense. Benchmarks such as RobustBench restrict entries to defenses where standardized attacks are considered reliable (for example, no randomness, no non-differentiable preprocessing, no detection).

3 Adaptive Attacks: Evaluating Defenses Properly

Definition 3.1 (Adaptive attack). An attack designed with full knowledge of the defense, including its preprocessing, randomness, detectors and training procedure, that optimizes an objective aligned with defeating that specific defense.

3.1 A recipe

Following Tramèr et al. (2020) and Carlini et al. (2019):

1. **Understand the defense.** Identify every component between input and decision: transformations, randomness, detectors, ensembles, abstention.
2. **Simplify.** Remove components that do not contribute to robustness and check whether the remaining core is still robust. Defenses often have one component doing the work.
3. **Define a loss that matches success.** If the defense also detects, success means *misclassified and not detected*: combine the classification margin with the detector score. If the defense abstains, abstention is not a correct answer unless the threat model says so.
4. **Make the loss optimizable.** Replace non-differentiable steps with BPDA, average over randomness with EOT, use logits or margin losses to avoid saturation, reparameterize iterative defenses (Chapter 5).
5. **Optimize well.** Many iterations, restarts, step-size schedules; verify convergence by plotting the loss.
6. **Try gradient-free attacks** as a check (Square, SPSA, decision-based).

3.2 Sanity checks

Checklist before believing a robustness number

- Accuracy decreases monotonically in ϵ and reaches (near) zero for large ϵ .
- Iterative attacks beat single-step attacks; white-box beats black-box and transfer.
- More iterations and restarts do not increase the success rate further (convergence).
- Targeted attacks toward each class succeed for large ϵ .
- The attack succeeds against the undefended base model with the same hyperparameters.
- Adversarial examples are verified: inside the threat set, valid domain, misclassified by the full defended pipeline (with fresh randomness).
- Clean accuracy is reported alongside robust accuracy.

Example 3.1 (Detector plus classifier). A defense pairs a classifier f with a detector $d(\mathbf{x}) \in [0, 1]$, rejecting inputs with $d(\mathbf{x}) > \tau$. Attacking only f finds examples that d flags, so the defense appears robust. The adaptive attack optimizes

$$\max_{\mathbf{x}' \in \mathcal{B}_p(\mathbf{x}, \epsilon)} \left[\max_{j \neq y} Z_j(\mathbf{x}') - Z_y(\mathbf{x}') \right] - \lambda \max(0, d(\mathbf{x}') - \tau + \kappa),$$

or equivalently treats the detector’s rejection as an extra class $K + 1$ of a combined model and requires the attack to avoid both y and $K + 1$. This construction bypassed ten published detectors (Carlini and Wagner, 2017).

4 Security Evaluations under Different Threat Models

A complete evaluation states which threat models were tested and which were not.

Knowledge. White-box evaluations upper-bound what weaker attackers can do; they are the reference. Black-box results are informative about deployment risk (queries needed, cost) but are not robustness evidence. Transfer results from multiple surrogates indicate whether the defense resists attackers without model access.

Multiple and unforeseen perturbations. Robustness to one ℓ_p ball does not transfer to others (Tramèr and Boneh, 2019). Evaluate several norms and their union (Maini et al., 2020), spatial transformations such as rotations and translations (Engstrom et al., 2019), perceptual threat models (Laidlaw et al., 2021), and natural corruptions such as noise, blur and weather (Hendrycks and Dietterich, 2019). Report results for the threat model used in training separately from the unseen ones.

Training-time threats. For poisoning defenses, evaluate with attacks that know the defense (for example, poisons crafted to evade spectral filtering), across poisoning rates and trigger types, and report clean accuracy and ASR after the defense.

Privacy threats. Use the strongest available membership inference (LiRA, Chapter 4) at low FPR, include canaries or worst-case records, and compare with DP guarantees when they exist.

System-level threats. Real attackers often do not compute gradients (Apruzzese et al., 2023): they exploit the pipeline (input validation, preprocessing, fallbacks, human review). A threat model for a deployed system should include those components, and the evaluation should state what is out of scope.

5 Interpreting Robustness Evaluation Results

Upper versus lower bounds. By Eq. (1), an empirical number is an upper bound that can only go down as attacks improve; a certified number is a lower bound. The gap between them measures how much we do not know.

Statistical uncertainty. Robust accuracy is a binomial proportion. With N test points and estimate $\hat{p}$, a 95% normal-approximation interval is

$$\hat{p} \pm 1.96\sqrt{\hat{p}(1 - \hat{p})/N}.$$

For $N = 1000$ and $\hat{p} = 0.55$ the half-width is about 3.1 percentage points, so differences of one or two points between defenses evaluated on 1000 examples are not meaningful. Variance across training seeds should also be reported.

Comparability. Results are comparable only with identical threat model (p , ϵ , input scaling), test subset, attack configuration, and the same handling of initially misclassified points. Check whether ϵ is in $[0, 1]$ or $[0, 255]$ units and whether normalization was applied before or after the perturbation.

What a number does not mean. 60% robust accuracy at ℓ_∞ , $\epsilon = 8/255$ does not mean the model is 60% secure in deployment. It means that, for this threat model and this attack suite, on this test distribution, 40% of inputs admit an adversarial example. It says nothing about larger perturbations, other perturbation types, poisoning, or privacy.

Minimum reporting standard

Threat model (goal, knowledge, capability); clean accuracy; robust accuracy with worst-case over an attack suite including AutoAttack and at least one adaptive attack; robustness curve over ϵ ; attack hyperparameters and convergence evidence; confidence intervals; code and model release.

6 Hands-On: Setting Up Robustness Benchmarks

6.1 Loading models and running AutoAttack

```
# pip install git+https://github.com/RobustBench/robustbench.git
# pip install git+https://github.com/fra31/auto-attack
import torch
from robustbench.utils import load_model
from robustbench.data import load_cifar10
from autoattack import AutoAttack

x, y = load_cifar10(n_examples=1000)
x, y = x.cuda(), y.cuda()

std = load_model("Standard", dataset="cifar10", threat_model="Linf").cuda().eval()
# pick any entry from the RobustBench CIFAR-10 Linf leaderboard
rob = load_model("<leaderboard-model-name>", dataset="cifar10",
                 threat_model="Linf").cuda().eval()

for name, model in [("standard", std), ("robust", rob)]:
    adversary = AutoAttack(model, norm="Linf", eps=8/255, version="standard")
    x_adv = adversary.run_standard_evaluation(x, y, bs=250)
    acc = (model(x_adv).argmax(1) == y).float().mean().item()
    print(name, "robust acc:", acc)
```

6.2 Robustness curve with per-example worst case

```
import numpy as np

def worst_case_robust(model, x, y, attacks):
    # attacks: list of callables (model, x, y) -> x_adv
    with torch.no_grad():
        robust = model(x).argmax(1) == y
    for atk in attacks:
        idx = robust.nonzero().squeeze(1)
        if len(idx) == 0: break
        xa = atk(model, x[idx], y[idx])
        assert (xa - x[idx]).abs().max() <= eps + 1e-6 # threat model check
        assert xa.min() >= 0 and xa.max() <= 1
        with torch.no_grad():
            robust[idx] &= model(xa).argmax(1) == y[idx]
    return robust.float().mean().item()

def ci95(p, n):
    h = 1.96 * np.sqrt(p * (1 - p) / n)
    return p - h, p + h

for eps in [0, 1/255, 2/255, 4/255, 8/255, 16/255, 32/255]:
    attacks = [lambda m, a, b: pgd_linf(m, a, b, eps, eps / 4, 50, loss="ce")[0],
               lambda m, a, b: pgd_linf(m, a, b, eps, eps / 4, 50, loss="cw")[0]]
    ra = worst_case_robust(rob, x, y, attacks)
    print(f"eps={eps*255:.0f}/255 RA={ra:.3f} CI={ci95(ra, len(x))}")
```

(pgd_linf is the Chapter 2 implementation; loss="cw" selects the margin loss.)

6.3 Suggested experiments

1. Evaluate a standard model and two RobustBench models with AutoAttack at $\epsilon = 8/255$ on 1000 points. Compare with your own PGD-20. Report the gap and its confidence interval.
2. Plot robustness curves from $\epsilon = 0$ to $32/255$. Check that they reach zero.
3. Take your fast-AT model from Chapter 5 at an epoch with catastrophic overfitting and run the sanity checklist. Which checks fail?
4. Wrap a robust model with a randomized input transformation (random resize and pad). Evaluate it with AutoAttack *standard*, AutoAttack *rand* (EOT), and a hand-written EOT-PGD with 20 samples per gradient. Compare.
5. Evaluate an ℓ_∞ -robust model under ℓ_2 ($\epsilon = 0.5$), rotations up to 30° , and CIFAR-10-C corruptions. Summarize in a table.

Exercises

1. Two attacks give 52% and 50% robust accuracy. Show that the per-example worst-case robust accuracy can be anywhere between 2% and 50%, and explain why it matters.
2. Design an adaptive attack for a defense that applies JPEG compression at a random quality factor before a classifier.
3. A paper reports 71.3% vs 70.1% robust accuracy for two methods on 500 test points with one seed each. Is the comparison meaningful? Compute the confidence intervals.
4. Explain why a certified accuracy higher than an empirical robust accuracy, for the same model and ϵ , indicates a bug.
5. Write a one-paragraph threat model and evaluation plan for a robustness claim about a malware detector, stating what is explicitly out of scope.

References

- Apruzzese, G., Anderson, H. S., Dambra, S., Freeman, D., Pierazzi, F., and Roundy, K. (2023). “real attackers don’t compute gradients”: Bridging the gap between adversarial ML research and practice. In *IEEE Conference on Secure and Trustworthy Machine Learning (SaTML)*, pages 339–364.
- Athalye, A., Carlini, N., and Wagner, D. (2018). Obfuscated gradients give a false sense of security: Circumventing defenses to adversarial examples. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, pages 274–283.
- Carlini, N., Athalye, A., Papernot, N., Brendel, W., Rauber, J., Tsipras, D., Goodfellow, I., Madry, A., and Kurakin, A. (2019). On evaluating adversarial robustness. *arXiv preprint arXiv:1902.06705*.
- Carlini, N. and Wagner, D. (2017). Adversarial examples are not easily detected: Bypassing ten detection methods. In *Proceedings of the 10th ACM Workshop on Artificial Intelligence and Security (AISec)*, pages 3–14.
- Croce, F., Andriushchenko, M., Sehwag, V., Debenedetti, E., Flammarion, N., Chiang, M., Mittal, P., and Hein, M. (2021). RobustBench: A standardized adversarial robustness benchmark. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*.
- Croce, F. and Hein, M. (2020a). Minimally distorted adversarial examples with a fast adaptive boundary attack. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, pages 2196–2205.
- Croce, F. and Hein, M. (2020b). Reliable evaluation of adversarial robustness with an ensemble of diverse parameter-free attacks. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, pages 2206–2216.
- Engstrom, L., Tran, B., Tsipras, D., Schmidt, L., and Madry, A. (2019). Exploring the landscape of spatial robustness. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, pages 1802–1811.
- Hendrycks, D. and Dietterich, T. (2019). Benchmarking neural network robustness to common corruptions and perturbations. In *International Conference on Learning Representations (ICLR)*.
- Kim, H. (2020). Torchattacks: A PyTorch repository for adversarial attacks. *arXiv preprint arXiv:2010.01950*.
- Laidlaw, C., Singla, S., and Feizi, S. (2021). Perceptual adversarial robustness: Defense against unseen threat models. In *International Conference on Learning Representations (ICLR)*.
- Maini, P., Wong, E., and Kolter, J. Z. (2020). Adversarial robustness against the union of multiple perturbation models. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, pages 6640–6650.
- Morris, J., Lifland, E., Yoo, J. Y., Grigsby, J., Jin, D., and Qi, Y. (2020). TextAttack: A framework for adversarial attacks, data augmentation, and adversarial training in NLP. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 119–126.
- Nicolae, M.-I., Sinn, M., Tran, M. N., Buesser, B., Rawat, A., Wistuba, M., Zantedeschi, V., Baracaldo, N., Chen, B., Ludwig, H., Molloy, I. M., and Edwards, B. (2018). Adversarial robustness toolbox v1.0.0. *arXiv preprint arXiv:1807.01069*.
- Rauber, J., Zimmermann, R., Bethge, M., and Brendel, W. (2020). Foolbox native: Fast adversarial attacks to benchmark the robustness of machine learning models in PyTorch, TensorFlow, and JAX. *Journal of Open Source Software*, 5(53):2607.

- Tramèr, F. and Boneh, D. (2019). Adversarial training and robustness for multiple perturbations. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- Tramèr, F., Carlini, N., Brendel, W., and Madry, A. (2020). On adaptive attacks to adversarial example defenses. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- Weng, T.-W., Zhang, H., Chen, P.-Y., Yi, J., Su, D., Gao, Y., Hsieh, C.-J., and Daniel, L. (2018). Evaluating the robustness of neural networks: An extreme value theory approach. In *International Conference on Learning Representations (ICLR)*.